

1 Roteiro Prático 02 — Retomada e Transição Parte 2: jQuery, Manipulação do DOM & LocalStorage

Objetivos

Objetivos da Aula (Parte 2 — jQuery & Dinâmica Frontend)

Ao final desta aula o estudante será capaz de:

- Retomar o layout estático construído no Bootstrap 5 e adicionar inteligência no lado do cliente com **jQuery**;
- Localizar e obter a biblioteca no portal oficial do **jQuery** (<https://jquery.com> e <https://code.jquery.com>);
- Incluir os scripts no final do <body> e organizar a lógica em arquivo JavaScript externo (app.js);
- Compreender o manipulador de inicialização segura `$(document).ready(function() { ... });`;
- Selecionar elementos do DOM com `$()` e escutar eventos de formulário e clique (`.on('submit')`, `.click()`);
- Aplicar efeitos visuais e animações fluidas na interface (`.fadeIn()`, `.slideToggle()`, `.addClass()`);
- Inserir e remover elementos dinamicamente na árvore DOM com `.append()`, `.val()` e `.remove()`;
- Persistir e recuperar dados no navegador utilizando o **LocalStorage** (`setItem`, `getItem`, `JSON.stringify`, `JSON.parse`);
- Implementar filtro de busca dinâmico em tempo real na tabela;
- Acompanhar o material através da apresentação de slides interativa da Parte 2 (<https://chameoandre.github.io/CST-Sistemas-para-internet-programacao-para-internet-2/retomada-de-atividades-pi2/parte-2-jquery/index.html>).

1.1 Introdução, Sites Oficiais e Organização em Arquivo Externo `app.js`

Bem-vindos à segunda parte da retomada prática da disciplina de **Programação para a Internet 2 (ProgWeb 2)**!

Nesta aula, transformamos a interface estática do Bootstrap 5 em um sistema ****dinâmico, interativo e funcional**** utilizando a biblioteca ****jQuery****.

1. **jQuery (Site Oficial & CDN):** No portal <https://jquery.com> e no servidor de CDN <https://code.jquery.com>, obtemos os links da versão minificada (`jquery-3.7.1.min.js`), incluída via tag `<script>` antes do fechamento do `</body>`;
2. **Organização Profissional em Arquivo Externo (`app.js`):** Adotamos como boa prática o desacoplamento do código JavaScript em um arquivo separado (`app.js`), inicializado com `$(document).ready()`;
3. **Acompanhamento Interativo:** O trabalho é guiado pelos slides da Parte 2 (`parte-2-jquery/index.html`) e pela prática na pasta `exercicio-retomada/`.

1.2 Conceitos Fundamentais: O que é o DOM (Document Object Model)?

Antes de manipularmos os elementos da página com o jQuery, é fundamental compreender o papel do **DOM (Document Object Model)**:

- **Significado da Sigla:** *Document Object Model* (Modelo de Objeto do Documento). É a representação em memória (estrutura em árvore) que o navegador cria ao ler o arquivo HTML, transformando cada tag em um objeto manipulável;
- **Responsabilidades Principais do DOM:**
 1. **Mapeamento Hierárquico:** Organiza a relação estrutural entre elementos pais e filhos (ex: `<form>` contendo `<input>`);
 2. **Interface de Interatividade:** Fornece comandos para ler valores, alterar textos, aplicar estilos CSS e escutar eventos do usuário (cliques e submissões);
- **Interação via JavaScript / jQuery:** O código JS consulta a árvore usando seletores como `$('#input-nome')`. A partir daí, executa operações dinâmicas: lê dados (`.val()`), altera textos (`.html()`), modifica cores (`.css()`) e reage a eventos (`.on('submit')`).

1.3 Sintaxe Básica do jQuery & O Comando `$(document).ready()`

Antes de manipularmos o formulário, precisamos compreender como as instruções são estruturadas e por que o método de inicialização é indispensável:

- **A Sintaxe Padrão `$(seletor).acao()`:**

- `$`: É o atalho para invocar a biblioteca jQuery;
- `seletor`: A string com o seletor CSS do elemento desejado (ex: `'#btn-salvar'`);
- `acao()`: O método que lê, modifica ou escuta o elemento (ex: `.val()`, `.html()`, `.on()`);
- **O que faz o `$(document).ready(function() { ... })`:** É a função de **inicialização segura**. Ela garante que qualquer código JavaScript/jQuery só seja executado **após o navegador ter concluído a leitura do HTML e a montagem da árvore do DOM na memória RAM**;
- **Por que temos que utilizá-lo (A causa do erro):** Se o script rodar antes de o HTML ser lido, o elemento (ex: `#form-cadastro`) ainda não existirá na memória. O jQuery retornará um objeto nulo/vazio e qualquer tentativa de escutar um clique ou submissão falhará silenciosamente;
- **Sintaxe Abreviada Equivalente:** A comunidade também utiliza a forma curta moderna: `$(function() { /* código seguro */ });`.

1.4 Metodologia Pedagógica Incremental: A Mesma Aplicação Base, Recursos Crescentes

Para garantir que o aprendizado seja fluido e contínuo, adotamos a **Metodologia Incremental**:

- **Estrutura Base Unificada:** Em vez de criar um código HTML novo do zero para cada exercício, mantemos a mesma página de **Cadastro de Clientes** criada na Parte 1 com Bootstrap 5;
- **Acúmulo de Superpoderes Passo a Passo:**
 1. **Exemplo 6 (Base):** Leitura de inputs com `.val()` e escrita em alerta com `.html()`;
 2. **Exemplo 7 (Feedback):** Remoção de `.d-none`, `.addClass('alert-success')` e piscar animado 2x com `.fadeOut().fadeIn()`;
 3. **Exemplo 8 (Estilos):** Troca dinâmica de cores de fundo e bordas via `.css({ ... })`;
 4. **Exemplo 9 (Efeitos):** Deslizamento de painel retrátil com `.slideToggle()` e `.slideDown()`;
 5. **Exemplo 10 (DOM Dinâmico):** Montagem de tabela dinâmica com `.append()` e remoção de linha pai com `.closest('tr').remove()`;
 6. **Exemplo 11 (Persistência):** Armazenamento permanente no navegador com `localStorage` e conversão `JSON.stringify/parse`.

1.5 Detalhamento Didático da Estrutura HTML Base Unificada & Exemplos 6 a 11

Para manter uma linha contínua de aprendizado sem perder a conexão entre os seletores, todos os exemplos da Parte 2 utilizam a **mesma estrutura HTML base** da aplicação de Cadastro de Clientes (Exemplo 5 do Bootstrap 5), incrementando-a passo a passo:

Estrutura HTML Base Unificada (Formulário, Alerta e Tabela):

```
<!-- Formulário de Cadastro (#form-cadastro) -->
<form id="form-cadastro" class="card p-3 mb-3">
  <input type="text" id="input-nome"
    class="form-control mb-2" placeholder="Nome">
  <input type="email" id="input-email"
    class="form-control mb-2" placeholder="E-mail">
  <button type="submit" id="btn-salvar"
    class="btn btn-primary">Salvar Cliente</button>
</form>

<!-- Caixa de Alerta para Feedback Visual (#caixa-mensagem) -->
<div id="caixa-mensagem"
  class="alert alert-info d-none"></div>

<!-- Tabela com Corpo Dinâmico (#tabela-corpo) -->
<table class="table table-striped">
  <thead>
    <tr>
      <th>Nome</th>
      <th>E-mail</th>
      <th>Ação</th>
    </tr>
  </thead>
  <tbody id="tabela-corpo"></tbody>
</table>
```

Abaixo está a progressão funcional realizada sobre a mesma estrutura pedagógica:

- **Exemplo 6 (Passo 1 — Leitura & Escrita de Campos):** *Estrutura Base Inicial:* Captura a submissão do #form-cadastro, impede o recarregamento com `e.preventDefault()`, lê os valores de #input-nome e #input-email com `.val()` e escreve na caixa #caixa-mensagem com `.html()`;
- **Exemplo 7 (Passo 2 — Animações Visuais & Classes CSS):** *Adicionado em relação ao Exemplo 6:* Incrementa a #caixa-mensagem removendo a classe `.d-none`, aplicando `.addClass('alert-success')` e fazendo o alerta piscar/pulsar de forma altamente visível encadeando `.fadeOut(150).fadeIn(150).fadeOut(150)`;
- **Exemplo 8 (Passo 3 — Troca Dinâmica de Cores e Estilos CSS):** *Adicionado em relação ao Exemplo 7:* Altera o estilo diretamente via código com `.css()`, modificando a cor de fundo do formulário e as bordas/cores da caixa de mensagem de forma dinâmica;

- **Exemplo 9 (Passo 4 — Efeitos Visuais de Deslizamento):** *Adicionado em relação ao Exemplo 8:* Requer a criação do botão #btn-toggle-ajuda e do painel retrátil #painel-ajuda (`style="display: none;"`). Explora o efeito `.slideToggle(300)` para alternar o painel de instruções e `.slideDown(400)` para a mensagem surgir deslizando;
- **Exemplo 10 (Passo 5 — Manipulação Dinâmica do DOM & Tabela):** *Adicionado em relação ao Exemplo 9:* Insere a nova linha `<tr>` com os dados e o botão de exclusão na #tabela-corpo com `.append()`. Ao clicar em Remover, exclui a linha pai com `$(this).closest('tr').fadeOut(300, function() { $(this).remove(); });`;
- **Exemplo 11 (Passo 6 — Persistência no LocalStorage com JSON):** *Adicionado em relação ao Exemplo 10:* Salva o array de clientes no navegador com `localStorage.setItem('lista_clientes', JSON.stringify(clientes))` e restaura a tabela ao recarregar a página com `JSON.parse()`.

1.6 Guia de Apoio: "Para Explorar Mais no jQuery"

Para dar suporte aos alunos na programação dinâmica, o quadro abaixo resume métodos essenciais do jQuery:

Opções Adicionais de jQuery (Seletores, Efeitos e Manipulação do DOM)	
Método / Seletor	Descrição e Uso no Código
<code>\$('.minha-classe')</code>	Seleciona todos os elementos do DOM que possuem uma classe CSS específica.
<code>\$(el).slideDown() / .slideUp()</code>	Animação fluida de expandir para baixo ou recolher para cima.
<code>\$(el).attr('disabled', true)</code>	Lê ou altera atributos HTML de um elemento (ex: desativar botões).
<code>\$(el).prepend(html)</code>	Insere novo conteúdo HTML no início do elemento selecionado.
<code>\$(el).empty() / .remove()</code>	Limpa todo o conteúdo interno ou remove o próprio elemento da página.

1.7 Parte 2 — Tarefa Prática: Programação Dinâmica em Arquivo Externo app.js

Atividade Prática – Dinâmica com jQuery e LocalStorage

Na pasta `exercicio-retomada/`, os estudantes deverão tornar a aplicação 100% dinâmica:

1. **Inclusão do jQuery:** Adicionar a tag `<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>` antes do `</body>`;

2. **Seleção e Eventos em `app.js`:** No arquivo externo `app.js`, capturar a submissão do formulário com `$('#formCadastro').on('submit');`
3. **Inserção Dinâmica no DOM:** Ler os campos com `$('#nome').val()` e adicionar a nova linha na tabela com `$('#tabela-body').append();`
4. **Persistência no LocalStorage (Exemplo 9):** Salvar a lista de registros com `localStorage.setItem()` e tratar a restauração dos dados ao abrir a página (F5).

1.8 Parte 3 — Desafio Extra: Filtro Dinâmico em Tempo Real

Desafio Extra – Filtro por Nome com jQuery

Como atividade de aprimoramento em sala:

1. Criar um campo de busca por nome acima da tabela (`$('#filtro-nome');`);
2. Implementar o evento `input` do jQuery para filtrar dinamicamente as linhas exibidas à medida que o usuário digita.

1.9 Checklist Final da Aula 2

Verificação Técnica Parte 2

Antes de finalizar o encontro, verifique se seu projeto atende aos critérios de aceitação:

O script do jQuery 3.7.1 está incluído e o arquivo `app.js` inicializa com `$(document).ready();`

Ao cadastrar um novo registro, a tabela é atualizada instantaneamente sem recarregar a página;

Ao pressionar F5 (Recarregar), os dados salvos no `LocalStorage` permanecem visíveis na tela.